

Project Overview Document (POD): Eisenhower Task Manager

"A production-ready task management application built on the Eisenhower Matrix framework, enabling users to prioritize tasks by urgency and importance while tracking planning/execution accuracy." — Eisenhower Task Manager Description

Project: Eisenhower Task Manager — Interactive Task Prioritization Application **Owner:** Cole Prather **Last Updated:** 2026-01-27 **Status:** Active (Production-Ready, Continuous Improvement)

1. What This Project Is

The Eisenhower Task Manager is a full-stack web application for personal productivity that implements the Eisenhower Matrix (2×2 framework of urgency vs. importance). Users add tasks, categorize them as urgent/not-urgent and necessary/not-necessary, and watch them automatically organize into four quadrants: Do First (urgent + necessary), Schedule (not urgent + necessary), Delegate (urgent + not necessary), and Eliminate (neither). The application is production-ready with multiple deployment options (browser-only or file-based persistence), comprehensive documentation, and a V2.0 feature set including task completion verification and planning/execution scoring.

The project combines pedagogical rigor with practical utility: the Eisenhower Matrix is a well-established decision framework, and this implementation makes it tangible and interactive. Users can add tasks manually, bulk-import to-do lists via AI parsing, mark tasks complete with quality/ease ratings, and visualize their planning accuracy through a scoring system that measures how well they estimated due dates. The application supports recurring tasks, customizable categories, detailed notes, and both matrix and list views.

The technical architecture is a modern React frontend (Vite build system) with an optional Express.js backend for file-based persistence. Single-file distribution (ZIP packages) includes shell scripts for cross-platform setup. Multiple deployment versions have been created (v1, v1-FIXED, v2) reflecting iterative refinement.

2. Main Objectives

Objective	Success Looks Like	Status
Implement Eisenhower Matrix framework with 4-quadrant task organization	Tasks automatically sorted into Do First / Schedule / Delegate / Eliminate quadrants based on urgency/necessity flags.	Complete
Enable manual task entry with full metadata (dates, categories, notes, recurrence)	Users can create tasks with: category, subcategory, assigned/due dates, recurring patterns, notes, quality/ease ratings.	Complete
Support bulk import of to-do lists via AI-powered parsing	"AI Input" modal accepts unstructured text, parses into structured tasks. Works with lists, notes, email text.	Complete
Provide dual view modes (Matrix and List) with full functionality in both	Matrix shows 4 quadrants with cards; List shows table with sortable columns. Both support all operations.	Complete
Track task completion with quality and ease ratings	Completion modal prompts for 1-5 star ratings on quality and ease. Data stored with completed task.	Complete
Implement Planning/Execution Score measuring estimation accuracy	Score formula: $(\text{due} - \text{completed}) / (\text{due} - \text{assigned})$. Color-coded visualization (red=late, green=early).	Complete
Support recurring tasks with pattern options (daily, weekly, monthly, yearly)	Recurring field with pattern selector. Used in future schedule forecasting (implied).	Complete
Enable customizable categories and subcategories	Settings interface allows creating/editing categories. Categories persistent across sessions.	Complete
Provide backup/export functionality for data	Export backup button generates JSON file. Import allows loading previous backups.	Complete

portability		
Support multiple deployment modes (browser-only and file-based)	localStorage mode for quick start; file-based mode with Express backend for persistence.	Complete

3. Current Status

Overall Assessment: The Eisenhower Task Manager is **production-ready and actively deployed** (v2.0.1). Core functionality is complete and well-tested. The application serves its primary purpose: helping users prioritize tasks using a proven decision framework. Remaining work is primarily code quality improvements (refactoring monolithic App.jsx), security enhancements (server validation), and advanced analytics features (trend analysis, category performance). The project is 85% toward a fully polished, enterprise-grade task manager.

What's Working:

- **Eisenhower Matrix fully functional:** Tasks correctly sort into 4 quadrants based on urgency × necessity flags. Quadrant labels clear (Do First, Schedule, Delegate, Eliminate). Drag-and-drop reordering works within quadrants.
- **Task management complete:** Create, read, update, delete all working. Metadata fields (category, subcategory, assigned date, due date, notes, quality/ease ratings) all functional.
- **Dual view system robust:** Matrix view shows visual 4-quadrant layout. List view shows table with sortable columns, filters, and inline editing. Switching between views preserves state.
- **Planning/Execution Score live:** calculateTaskScore() function working correctly. Color-coded visualization (red-to-green gradient). Edge cases handled (incomplete tasks, null dates, same assigned/due dates).
- **Completion verification modal:** Prompts for quality (1-5 stars) and ease (1-5 stars) ratings when marking tasks complete. Validation prevents submission without both ratings.
- **Recurring task support:** Pattern options include once, daily, weekly, monthly, yearly. Data structure supports future auto-generation of recurring instances.
- **AI bulk import:** "AI Input" modal accepts text, parses into structured tasks, supports common list formats. Accuracy depends on input clarity.
- **Data persistence options:** localStorage for browser-only, file-based (JSON) with Express

backend for server mode. Both fully tested.

- **Backup/export working:** Export generates JSON file with full task history. Import loads previous backups with validation.
- **Responsive design:** Works on desktop, tablet, mobile. Controls adapt to viewport. Dark mode implemented.
- **Documentation excellent:** Multiple guides (QUICKSTART.md, SCORE_FEATURE_GUIDE.md, UPDATES_MODIFICATIONS_LIST.md). Clear onboarding for new users.

What's Not Working:

- **Code organization monolithic:** App.jsx is 70 KB / 1,755 lines. All components, logic, utilities in single file. Difficult to maintain, test, and extend. Refactoring needed.
- **Server input validation absent:** No validation on API endpoints in server.js. Could accept malformed data. No payload size limits. Error handling minimal.
- **Analytics/insights limited:** Score visible on individual tasks, but no trend analysis. Missing: "Average score this month," "Best category," "Worst week," "Improvement over time."
- **Recurring task generation incomplete:** Data structure supports recurring patterns, but auto-generation of future instances not implemented. Users must manually create instances.
- **Search functionality missing:** Can't search tasks by name, category, or date range. Important as task list grows.
- **Task filtering limited:** Can filter by category, but not by status (urgent, necessary), completion date, or score range.
- **Keyboard shortcuts absent:** No vim-style or productivity shortcuts (e.g., 'n' for new task, 'c' for complete).
- **Mobile touch optimization incomplete:** Drag-and-drop works on desktop but not reliably on mobile. Buttons could be larger.
- **No sync across devices:** Each device/browser stores separate copy. No cloud sync or multi-device support.
- **Testing coverage unknown:** No visible test suite. Manual testing only (implied).

Recent Progress:

- **January 24, 2026:** v2.0 released with Planning/Execution Score feature. Completion

modal added with quality/ease ratings. Score formula implemented and visualized.

- **January 24, 2026:** Comprehensive documentation released (SCORE_FEATURE_GUIDE.md, UPDATES_MODIFICATIONS_LIST.md). Feature prioritization and roadmap outlined.
- **January 23-24, 2026:** Multiple versions created and tested (v1, v1-FIXED, v2) reflecting iterative bug fixes and feature additions.
- **Active user data:** Tasks-2026-01-27.json shows real usage with mix of completed and pending tasks across Career (Physics, Dynamics) and Personal (Car, Finance) categories.

4. Issues and Hurdles

Active Issues

Issue	Why It Matters	What We're Doing About It
App.jsx monolithic (1,755 lines)	Difficult to maintain, test, debug, extend. High cognitive load. Risk of regression.	Refactor into modular components: CompletionModal, TaskForm, MatrixView, ListView, StatsBar, plus custom hooks (useTaskManager, useStorage) and utilities. Target: App.jsx < 300 lines.
No server input validation	Malformed data could break application. No security against bad payloads. No error messages for clients.	Add validation middleware in server.js. Validate task structure, settings structure, payload size. Return 400 with clear error messages.
Analytics completely missing	Score is computed but not analyzed. Can't see trends, patterns, or improvement. Users don't know if they're getting better at planning.	Implement: (1) Daily average score, (2) Category-wise breakdown, (3) Weekly trends, (4) Score distribution histogram. Requires historical data aggregation.
Recurring task generation incomplete	Recurring pattern data stored but not used. Users can't rely on app to generate future instances automatically.	Implement background job or user action that generates instances based on pattern. E.g., "Generate next 90 days" button or auto-generate on load.
Search functionality absent	As task list grows, finding specific tasks becomes tedious. No way to filter by	Add search input with real-time filtering across task name, category, notes. Consider indexed search for large lists.

	name or date.	
No keyboard shortcuts	Productivity power users expect vim/emacs-style shortcuts. Currently keyboard-hostile.	Implement: 'n' = new task, 'c' = complete task, '/' = search, 'v' = toggle view. Use keyboard-shortcut library (mousetrap or similar).
Mobile drag-and-drop unreliable	Quadrant reordering works great on desktop, but flaky on mobile. Touch events not properly handled.	Test touch events on real mobile devices. Use react-beautiful-dnd with touch-backend. Consider swipe-to-move alternative on mobile.
No multi-device sync	Tasks only sync within single browser/device. If user switches to phone, data is separate.	Requires backend. Implement: (1) User accounts / auth, (2) Sync API endpoints, (3) Conflict resolution strategy (last-write-wins? manual merge?). Large scope.

Structural Hurdles

Monolithic architecture grows linearly with features: Each new feature (score, ratings, AI input) adds to App.jsx. Without refactoring now, future features become increasingly difficult. Code debt accumulates. The time to add a new feature will grow from hours to days.

Single-file deployment limits extensibility: ZIP packages are convenient for initial distribution, but maintaining multiple versions (v1, v1-FIXED, v2) is error-prone. Version control is manual. No easy way to patch production.

Analytics require historical data infrastructure: Current system computes score on-demand from task dates. To implement trend analysis, need to (1) track historical scores, (2) aggregate by time period, (3) compute moving averages. Requires new storage layer.

Validation at boundary is weak: While the app itself doesn't corrupt data, there's no safeguard against: - Importing malformed JSON - Submitting empty tasks - Invalid date ranges (due before assigned) - Unreasonable recurrence patterns

Testing gap: No visible test suite. Critical paths (task creation, completion, scoring) should have unit + integration tests. Without tests, refactoring is risky.

Documentation implementation details: Feature guides are excellent for users, but lack technical implementation details. New developers would struggle to understand scoring formula implementation, storage strategy, or component architecture.

5. Goals and Next Steps

Immediate Priorities (Next 2-4 Weeks)

1. **Refactor App.jsx into modular components:** Extract TaskForm, MatrixView, ListView, CompletionModal, StatsBar into separate files. Create useTaskManager and useStorage hooks. Create taskHelpers.js and dateHelpers.js utilities. Target: App.jsx < 300 lines.
2. **Add server input validation:** Validate task and settings data structures in server.js. Add payload size limits (e.g., max 5MB per request). Return 400 with descriptive error messages. Add request logging.
3. **Implement basic analytics dashboard:** Display current stats: total tasks, completed %, average score, best category, hardest category. Update live as tasks complete.
4. **Write unit tests for critical paths:** Test calculateTaskScore(), task CRUD operations, recurring pattern logic, AI parser. Target 80% coverage of core logic.
5. **Add search and basic filtering:** Implement text search across task names/categories. Add filters for status (completed/pending), category, date range, score range.

Upcoming Milestones

Milestone	Target Date	Dependencies/Notes
App.jsx refactoring complete	2026-02-07	Extract all components into src/components/. Create hooks in src/hooks/. Create utilities in src/utlis/.
Server validation implemented	2026-02-10	Add middleware function for task/settings validation. Test with malformed inputs. Document error responses.
Basic analytics dashboard live	2026-02-14	Display summary stats on main view or sidebar. Update live. Requires aggregation logic.
Unit test coverage at 80%	2026-02-21	Write Jest tests for calculateTaskScore, CRUD operations, recurring logic. Mock storage layer.
Search and filtering functional	2026-02-28	Implement text search input. Add filter dropdowns. Test with large task lists (100+).
Recurring task auto-generation	2026-03-07	Implement "Generate next 90 days" feature or background generation. Consider edge cases (DST, month boundaries).
Production v2.1 release	2026-03-14	Tag release, update documentation, publish to website.

Open Questions

- **Should recurring task generation be automatic or user-initiated?** Automatic (on load) risks creating 100s of tasks. User-initiated (button) gives control but requires discipline. Hybrid (prompt on first recurring task)?
- **What's the target deployment: web app, Electron desktop, React Native mobile?** Currently web-only. Mobile port would significantly increase scope but increase utility.
- **Should there be a cloud sync option, or keep as local-first?** Cloud adds complexity and ongoing backend costs, but solves multi-device problem. Can defer to v3.0.
- **How deep should analytics go?** Basic stats (daily average, category breakdown)? Or

advanced (Lyapunov exponents for chaos-era tasks, Bayesian prediction of completion delays)? Start simple, let data inform future.

- **Should there be a public/shareable task board feature?** Users could share their Eisenhower matrix or task list. Interesting for accountability. Low priority.

6. Timeline

Start Date: ~Late 2025 (initial conception based on Dec 26 Excel file) **Target Completion (v2.1 Production):** 2026-03-14 **Current Projection:** On track; realistic release ~2026-03-21

Key Phases

Phase	Description	Timeframe	Status
Design & Prototyping	Eisenhower Matrix framework, UI mockups, feature prioritization	2025	Complete
Core Development	Task CRUD, matrix view, list view, basic UI	2025	Complete
V1.0 Release	Initial production version with localStorage	Jan 2026	Complete
V1 Bug Fixes	Address edge cases, improve stability (v1-FIXED)	Jan 2026	Complete
V2.0 Features	Planning/Execution Score, completion verification, ratings	Jan 2026	Complete
Code Refactoring	Modular architecture, component extraction, hook creation	Feb 2026	In Progress
Quality Improvements	Validation, testing, analytics, search/filtering	Feb–Mar 2026	Pending
Production Release (v2.1)	Full documentation, polished UI, security hardening	Mar 2026	Pending

Schedule Risks

- **Refactoring could uncover bugs:** When breaking monolithic App.jsx into components, risk of introducing regressions. Mitigation: write tests before refactoring (golden tests of current behavior).
 - **Analytics requires new data model:** Adding historical score tracking requires migration path for existing data. Need to be careful not to lose user data.
 - **Recurring task generation complexity:** Edge cases (month boundaries, leap years, DST transitions) are subtle. Testing required.
 - **No critical blockers, but scope could expand:** If user feedback requests multi-device sync or mobile app, timeline extends significantly.
-

7. Key Stakeholders and Resources

Core Team: Cole Prather (Designer, developer, product owner)

Decision Makers: Cole Prather

Intended Users: - Personal productivity enthusiasts - Students managing coursework (implied by task data) - Professionals using Eisenhower Matrix for work/life balance - Educators teaching time management (potential)

Technical Stack: - **Frontend:** React 18, Vite (build system), CSS (styling) - **Backend:** Express.js (optional, for file-based mode) - **Data Storage:** localStorage (browser) or JSON files (server) - **Deployment:** ZIP packages with shell scripts (cross-platform) - **Distribution:** Direct download + embedded in website

Deployment Platforms: - Desktop (Windows, Mac, Linux via Node.js) - Web (Vite dev server or production build) - Browser-only (single HTML + JS)

Budget/Resources: - Primarily sweat equity (Cole's development time) - No external hosting costs (local deployment) - No external dependencies beyond npm packages - Version management via ZIP snapshots (v1, v1-FIXED, v2)

8. Context and References

Background Reading: - [QUICKSTART.md](#) — 2-minute setup guide - [SCORE FEATURE GUIDE.md](#) — Planning/Execution Score explanation - [CHANGELOG V2.md](#) —

v2.0 release notes with completion verification - [UPDATES MODIFICATIONS LIST.md](#) —
Detailed roadmap with priority levels - [Task Data \(Jan 27\)](#) — Real usage example

Related Projects: - Dynamical Systems Laboratory (2026) — Similar interactive learning tool - ECDO Watch (2026) — Time-series data visualization - Cash Bubble Hypothesis (2026) — Decision-making framework (complements task prioritization)

Versions: - **v1.0:** Initial release with matrix view, list view, task CRUD - **v1-FIXED:** Bug fixes and stability improvements - **v2.0.1:** Planning/Execution Score feature, completion verification modal

Project Repository: - Local: `C:\Users\Cole\Dropbox\Website\projects\eisenhower-task-manager\` - Deployment formats: ZIP packages (v1, v1-FIXED, v2) - Source code: `eisenhower-task-manager-v2/eisenhower-app/src/`

Synthesis & Assessment

The Eisenhower Task Manager is a **well-conceived, production-ready application** that successfully implements a proven decision framework (Eisenhower Matrix) in an interactive, user-friendly form. The v2.0 release with Planning/Execution Score and completion verification represents maturity: the application now provides both a framework for making decisions (which quadrant?) and feedback on decision accuracy (how well did I estimate?).

Strengths:

- **Clear purpose:** Solves a real problem (task prioritization) using a proven framework (Eisenhower Matrix).
- **Feature completeness:** Core features are all there—task management, categorization, persistence, visualization, metrics.
- **Excellent documentation:** Multiple guides explaining features, scoring, and setup. Low barrier to entry.
- **Thoughtful UX:** Dual views (matrix + list), completion modal with ratings, color-coded scoring. Intentional design.
- **Pragmatic deployment:** ZIP packages with shell scripts enable cross-platform setup without external dependencies.
- **Active iteration:** Multiple versions show refinement and responsiveness to feedback.

Near-term priorities:

1. Refactor monolithic App.jsx into modular components (code quality)
2. Add server-side input validation (security)
3. Implement basic analytics dashboard (value)
4. Write unit tests for critical paths (reliability)
5. Add search and filtering (usability)

Long-term evolution:

- Recurring task auto-generation (quality of life)
- Multi-device sync via cloud backend (scaling)
- Mobile app (accessibility)
- Advanced analytics and trend analysis (insights)

Realistic assessment: The Eisenhower Task Manager is **2–3 weeks from v2.1 production-ready release** assuming focused effort on refactoring and validation. All features work. Remaining work is code quality, security, and analytics—important but not blocking current users. The application is suitable for personal and small-team use today; enterprise deployment would require backend hardening and user management.

This document provides strategic orientation and assessment. Detailed implementation tasks and code specifications belong in repository issues and project management systems.